In general terms, *serialization* means converting a high-level representation such as a Java `Object`—like a `String`—into a sequence of bytes. In an HTTP call, the default sequence of bytes today is JSON, which is, by default, UTF-8 encoded. In Akka HTTP, this is achieved using the library `akka-http-spray-json`, which uses the library `spray-json`.

Akka HTTP marshallers are tricky to understand but easy to use. We focus on how to use them, and to do that, you need to understand that they work by implicit resolution. *Implicit* means marshallers must be defined with the `implicit` keyword and available to the class that uses them through its imports, variables, or methods. To transform from class `A` to class `B`, you need a `Marshaller[A,B]` in the scope.

For example, to serialize a `String` into a JSON format, import `SprayJsonSupport._` and `DefaultJsonProtocol._` as follows:

```
import akka.http.scaladsl.marshallers.sprayjson.SprayJsonSupport._
import spray.json.DefaultJsonProtocol._
```

`spray-json` provides such a marshaller through `akka-http-spray-json` and can directly convert *basic* classes such as `String`, `Int`, `Byte`, `Option`, `List`, and others (see https://github.com/spray/spray-json#jsonprotocol). If the class is not a basic class, you must provide a formatter in the implicit resolution:

```
import spray.json.RootJsonFormat

final case class Cargo(kind: String, size: Int)
```